



Tecnológico Nacional de México Instituto Tecnológico
de Tijuana

Materia: Gestión de proyectos de software

Grupo: SC7C

Carrera: Sistemas Computacionales

Periodo: Agosto - Diciembre 2026

Docente: Maria Guadalupe Rodriguez Lopez

Alumnos:

González Cristóbal Omar

Garcia Luna Ricardo Daniel

Cuevas Marquez Pablo Angel

Perez Flores Andres Manuel

Unidad: 2

Actividad: Proyecto integrador / ETAPA I - Gestión de
calidad - 100%

Fecha: 06/10/2026

1. Nombre del proyecto

Nexus PC:

Sistema de venta de componentes de computadora en línea, con verificación de compatibilidad entre piezas.

Nexus PC es una tienda ficticia creada con fines académicos. No existe un proveedor real asociado: el catálogo de prueba (aproximadamente 30 productos) se captura con datos técnicos tomados de fichas públicas de los fabricantes y con precios de referencia.

2. Descripción de la problemática

Cada vez más personas compran componentes de computadora por internet, ya sea para armar un equipo nuevo o para mejorar uno existente. Sin embargo, una computadora solo funciona si sus piezas son compatibles entre sí: el procesador debe coincidir con el socket de la tarjeta madre, la memoria RAM debe ser del tipo que la tarjeta soporta (por ejemplo DDR4 o DDR5), la fuente de poder debe ofrecer la potencia suficiente y el gabinete debe aceptar el tamaño de la tarjeta madre.

Hoy quien compra suele revisar fichas técnicas en varios sitios para confirmar estas condiciones, y un error implica comprar piezas que no se pueden usar juntas, con la pérdida de tiempo y dinero que eso representa. Para quien es nuevo en el tema, la tarea resulta confusa.

3. Justificación

Un sistema que muestre el catálogo, sus características y precios facilita la compra tanto al cliente como al vendedor. Si además ayuda a detectar piezas incompatibles antes de pagar, reduce los errores de compra y vuelve más sencilla la experiencia de quien arma su primera computadora.

Desde la perspectiva de la materia, el proyecto permite aplicar un plan de calidad con requerimientos medibles, métricas, pruebas y control de cambios sobre un sistema completo con base de datos, servidor y página web, usando herramientas gratuitas y de uso extendido en la industria.

4. Usuarios a quienes está dirigido

Perfil	Descripción	Necesidad principal
Comprador que arma su primera PC	Persona con poco conocimiento técnico que elige todas las piezas.	Saber qué piezas son compatibles y no equivocarse al comprar.
Comprador que mejora un equipo	Persona que ya tiene una computadora y quiere cambiar o agregar piezas.	Encontrar componentes que funcionen con lo que ya tiene o está eligiendo.
Personal de la tienda	Encargado de consultar los pedidos registrados.	Contar con pedidos completos y ordenados en la base de datos.

5. Objetivo general

Desarrollar una base de datos de productos y una página web que permita consultar componentes de computadora, armar un carrito, verificar la compatibilidad entre piezas y registrar una compra sencilla, cumpliendo criterios de calidad definidos y medibles.

Objetivos específicos

- Diseñar e implementar en SQL Server la base de datos de categorías, productos, pedidos y detalle de pedidos.
- Diseñar con HTML y CSS una interfaz clara para el catálogo y el carrito.
- Programar una API con Node.js y Express que conecte la página con la base de datos.
- Implementar el registro de pedidos al confirmar una compra.
- Aplicar pruebas, métricas y control de cambios durante todo el desarrollo.

6. Alcance inicial

El alcance se clasifica por prioridad. Lo **esencial** se compromete para la entrega final; lo **deseable** se desarrolla en el orden indicado mientras el tiempo lo permita; lo **fuera de alcance** se documenta como trabajo futuro.

Prioridad	Funcionalidad
Esencial	Catálogo por categorías; detalle de producto; carrito de compras; vista rápida del carrito al pasar el cursor (o tocar); registro del pedido y su detalle en la base de datos.
Deseable 1	Verificación de compatibilidad entre componentes del carrito, con reglas mínimas (procesador-tarjeta madre, RAM-tarjeta madre, fuente-consumo, gabinete-tarjeta madre).
Deseable 2	Paquetes de componentes predefinidos (videojuegos, productividad, simulación) con piezas compatibles entre sí.
Deseable 3	Rendimiento tentativo en versión simplificada: índice relativo (1 a 10) por procesador y tarjeta de video, mostrado por tipo de uso, con aviso de que es una estimación.
Fuera de alcance	Pasarela de pago real; registro e inicio de sesión; panel de administración; envíos y seguimiento; FPS por videojuego y comparación por programa específico; despliegue en servidor de producción.

Supuestos y restricciones: el sistema se ejecuta en entorno local; el catálogo es de aproximadamente 30 productos en siete categorías (procesadores, tarjetas madre, memoria RAM, almacenamiento, tarjetas de video, fuentes de poder y gabinetes); el equipo son cuatro integrantes con unas 6 horas semanales cada uno; la entrega final y presentación son el 25 de noviembre de 2026.

7. Requerimientos funcionales principales

ID	Requerimiento	Prioridad	Criterio de aceptación
RF-01	El sistema muestra el catálogo de productos organizado por categorías.	Esencial	Cada una de las 7 categorías lista sus productos con nombre, marca, precio e imagen.
RF-02	El usuario puede ver el detalle de un producto.	Esencial	Se muestran precio, marca, descripción, especificaciones y disponibilidad.
RF-03	El usuario puede agregar, quitar y cambiar la cantidad de productos en el carrito.	Esencial	Las tres acciones actualizan el carrito sin recargar la página.
RF-04	El carrito calcula subtotales y total.	Esencial	El total coincide con la suma de precio por cantidad en el 100 % de las pruebas.
RF-05	Al pasar el cursor (o tocar) el área del carrito se muestra la lista de artículos agregados, y se oculta al salir.	Esencial	La lista aparece y desaparece correctamente en Chrome y Edge, y se abre con un toque en pantallas táctiles.

RF-06	Al confirmar la compra se registra el pedido y su detalle en la base de datos.	Esencial	Cada pedido guardado contiene todos sus productos, cantidades y total correctos.
RF-07	El sistema valida el stock y lo descuenta al confirmar el pedido.	Esencial	No se puede confirmar una cantidad mayor al stock; el stock nunca queda negativo.
RF-08	Botón "Compatibilidad": el usuario elige un componente de su carrito y el sistema muestra los productos por categoría, con palomita si son compatibles y equis (al final de la lista) si no lo son. Los incompatibles se pueden agregar de todos modos.	Deseable 1	Las 4 reglas mínimas se cumplen en el 100 % de los casos de prueba definidos; los incompatibles quedan al final y siguen siendo seleccionables.
RF-09	El sistema muestra paquetes predefinidos relacionados con el componente recién agregado.	Deseable 2	Ningún paquete contiene piezas incompatibles según las reglas de RF-08.
RF-10	El sistema muestra un rendimiento tentativo del carrito por tipo de uso (ofimática, videojuegos, edición 3D, navegación).	Deseable 3	Se muestra siempre con la leyenda de que es una estimación; el componente más débil determina el resultado.

8. Requerimientos no funcionales

ID	Categoría	Requerimiento	Forma de verificarlo
RNF-01	Rendimiento	El catálogo carga en 2 segundos o menos en entorno local con 30 productos.	Medición con las herramientas del navegador (pestaña Red).
RNF-02	Usabilidad	Agregar un producto al carrito requiere como máximo 3 clics desde el catálogo; la página es legible desde 360 px de ancho.	Prueba manual con lista de verificación y pruebas en distintos tamaños de pantalla.
RNF-03	Seguridad	Todas las consultas a la base de datos son parametrizadas; usuario y contraseña de la base de datos van en un archivo .env que no se sube al repositorio.	Revisión de código y revisión del archivo.gitignore.
RNF-04	Fiabilidad	El registro de un pedido es "todo o nada": si falla una parte, no se guarda un pedido incompleto.	Prueba provocando un error durante el registro.
RNF-05	Integridad de datos	La base de datos usa llaves primarias, llaves foráneas y restricciones (precio y stock no negativos).	Revisión del script SQL y pruebas de inserción inválida.
RNF-06	Compatibilidad	El sistema funciona en las versiones actuales de Chrome, Edge y Firefox.	Ejecución de la lista de pruebas en cada navegador.
RNF-07	Mantenibilidad	El código se organiza en carpetas por capa (base de datos, API, página), con nombres consistentes y commits descriptivos.	Revisión de pull requests y de la estructura del repositorio.

9. Criterios de calidad.

Los criterios se basan en el modelo de calidad del producto ISO/IEC 25010. Se eligieron las características que aplican a Nexus PC y se define para cada una qué significa y cómo se mide.

Característica	Qué significa en Nexus PC	Criterio medible
Adecuación funcional	El sistema hace lo que se prometió y lo hace correctamente.	Al menos 90 % de los requerimientos esenciales cumplidos en la entrega final; totales y stock correctos en el 100 % de las pruebas.
Eficiencia de desempeño	La página responde con rapidez.	Catálogo en 2 s o menos; compatibilidad en 3 s o menos (RNF-01).
Usabilidad	Un usuario nuevo puede comprar sin ayuda.	Un usuario ajeno al equipo completa una compra sin asistencia en 3 minutos o menos.
Fiabilidad	Los datos y pedidos se guardan sin errores ni pérdidas.	0 pedidos incompletos y 0 defectos críticos abiertos al momento de la entrega.
Seguridad	Los datos y la base de datos están protegidos a nivel básico.	100 % de las consultas parametrizadas; ninguna credencial en el repositorio.
Mantenibilidad	El equipo puede entender y modificar el código fácilmente.	Todo cambio llega por pull request revisado por otro integrante; cada archivo vive en su carpeta correspondiente.

10. Metodología de trabajo

El equipo trabaja con **Scrum simplificado**, adecuado para un equipo de cuatro personas y un semestre corto, porque permite entregar avances pequeños, ajustar el rumbo cada semana y dejar evidencia clara del progreso.

Elemento	Cómo se aplica
Sprints	Duración de una semana, con meta clara para cada uno.
Backlog	Lista priorizada de historias de usuario en GitHub Projects (tablero con columnas Pendiente, En proceso, En revisión y Terminado).
Reunión semanal	Reunión corta (unos 20 minutos) para revisar avances, bloqueos y plan de la semana; se deja nota breve en el repositorio.
Definición de "terminado"	Una tarea está terminada cuando cumple su criterio de aceptación, fue probada por quien la desarrolló, fue confirmada por el responsable de pruebas y calidad, y su pull request fue revisado por otro integrante.
Pruebas	Quien desarrolla una función o cambio la prueba primero; después Andrés, como responsable de pruebas y calidad, la prueba de nuevo para confirmarla. Si la tarea es de Andrés, la confirma otro integrante.

11. Integrantes y responsabilidades

Integrante	Rol principal	Responsabilidades	Apoyo
Daniel	Líder del proyecto y base de datos	Coordinar al equipo y las reuniones; aprobar cambios; diseñar el modelo de datos, crear las tablas y cargar los datos de prueba.	API (con Angel)
Angel	Backend (API)	Configurar Node.js y Express; conexión a SQL Server; rutas de productos, pedidos y compatibilidad.	Base de datos (con Daniel)
Omar	Frontend y diseño	Diseñar y programar el catálogo, el carrito y la vista rápida; integrar la página con la API.	Pruebas de usabilidad
Andrés	Documentación y responsable de pruebas y calidad	Redactar y mantener el plan de calidad, el registro de cambios y el manual; confirmar mediante prueba cada función o cambio que termina su autor; llevar las métricas.	Frontend (con Omar)

Responsabilidades compartidas: todos hacen commits propios, revisan al menos un pull request de otro integrante por semana, ejecutan pruebas y participan en la presentación final.

12. Herramientas y tecnologías propuestas

Herramienta	Uso en el proyecto
JavaScript	Lenguaje único del proyecto, tanto en la página (carrito, interacción) como en el servidor.
Node.js y Express	Entorno de ejecución del servidor y framework para las rutas de la API.
mssql y dotenv	Conexión de Node.js con SQL Server y manejo de credenciales fuera del código.
SQL Server y SSMS	Sistema gestor de base de datos y herramienta gráfica para crear tablas, ejecutar scripts y revisar datos.
HTML y CSS	Estructura y diseño visual del catálogo, el carrito y la vista de compatibilidad.
Visual Studio Code	Editor de código.
Git y GitHub	Control de versiones, issues, pull requests y tablero del proyecto (GitHub Projects).
Navegadores (Chrome, Edge, Firefox)	Ejecución de pruebas y medición de tiempos de respuesta.

Se eligió JavaScript frente a C# con ASP.NET Core y PHP por tener un solo lenguaje en todo el proyecto, curva de aprendizaje menor, costo cero y amplia documentación.

13. Métricas e indicadores de calidad

Las métricas se calculan al cierre de cada sprint y se registran en una tabla de seguimiento dentro del repositorio. El responsable de llevarlas es Andrés.

ID	Indicador	Fórmula	Meta	Frecuencia
M-01	Cumplimiento de requerimientos	Requerimientos esenciales cumplidos / requerimientos esenciales totales × 100	90 % o más al cierre; 100 % de los esenciales en la entrega final	Cada sprint
M-02	Funcionalidades terminadas	Historias de usuario terminadas / historias planeadas en el sprint × 100	80 % o más por sprint	Cada sprint
M-03	Pruebas satisfactorias	Pruebas aprobadas / pruebas ejecutadas × 100	95 % o más antes de la entrega	Cada sprint
M-04	Errores detectados	Número de defectos registrados, clasificados en crítico, mayor y menor	Registro completo; 0 críticos abiertos al entregar	Cada sprint
M-05	Densidad de errores corregidos	Defectos cerrados / defectos registrados × 100	85 % o más al cierre; 100 % de los críticos	Cada sprint
M-06	Tiempo de respuesta	Tiempo de carga del catálogo y de la consulta de compatibilidad (segundos)	Catálogo 2 s o menos; compatibilidad 3 s o menos	Antes de la entrega
M-07	Participación del equipo	Commits y pull requests revisados por integrante	Todos con actividad cada semana	Semanal
M-08	Cambios controlados	Cambios con solicitud aprobada / cambios realizados al alcance × 100	100 %	Cada sprint

Clasificación de defectos: **crítico** (impide comprar o pierde datos), **mayor** (falla una función pero hay alternativa) y **menor** (detalle visual o de texto).

14. Mecanismo para registrar y controlar cambios

Todo cambio al alcance, a los requerimientos o al diseño sigue este proceso, con GitHub como evidencia:

Paso	Descripción
1. Solicitud	Cualquier integrante abre un issue con la etiqueta “solicitud de cambio” usando el formato de abajo.
2. Análisis	El líder y el responsable del área afectada estiman el impacto en tiempo, base de datos, API, interfaz y pruebas.
3. Decisión	El líder aprueba o rechaza con el acuerdo de al menos otro integrante. Se anota el motivo en el issue.
4. Implementación	Se trabaja en una rama propia y se integra mediante pull request revisado por otro integrante.
5. Registro	Se actualiza el registro de cambios con versión y fecha, y se ajustan los documentos afectados.

Formato de solicitud de cambio

Campo	Contenido
ID y fecha	CAM-001, CAM-002... y fecha de la solicitud.
Solicitante	Nombre del integrante.
Descripción y motivo	Qué se quiere cambiar y por qué.

Impacto	Tiempo estimado, partes afectadas (BD, API, interfaz, pruebas) y riesgo.
Decisión	Aprobado, rechazado o pospuesto; quién decidió y por qué.
Estado	Pendiente, en desarrollo o cerrado.

Reglas adicionales

- Después del **11 de noviembre** (congelamiento de funciones) no se aceptan funciones nuevas, solo corrección de errores.
- Una funcionalidad “deseable” pasa a “esencial”, o una esencial se retira, únicamente mediante solicitud de cambio aprobada.
- Las versiones del documento y del sistema se numeran así: v0.x durante el desarrollo y v1.0 en la entrega final.
- Los mensajes de commit describen qué se cambió y, cuando aplique, incluyen el ID del cambio (por ejemplo, “CAM-003: agrega regla de compatibilidad de RAM”).

Registro de cambios

Versión	Fecha	ID	Descripción	Autor	Estado
0.1	3 oct 2026	–	Línea base: Plan de Calidad inicial con alcance clasificado por prioridad.	Equipo	Cerrado